

Mini Full Stack Web Application

Here's a complete mini full-stack Task Manager web application using MERN stack (MongoDB, Express.js, React, Node.js) with authentication, CRUD operations, task categorization, status updates, and search functionality.

Tech Stack Used

Frontend: React.js, React Router, Axios, Bootstrap

Backend: Node.js, Express.js, MongoDB (Mongoose)

Authentication: JSON Web Token (JWT), bcrypt.js

Database: MongoDB (local or cloud – MongoDB Atlas)

1.Backend (Node.js + Express.js + MongoDB)

- Install Dependencies

```
npm init -y
```

```
npm install express mongoose bcryptjs jsonwebtoken dotenv cors body-parser
```

Code :

Create Backend Files

1.1 server.js

```
require('dotenv').config();

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const authRoutes = require('./routes/authRoutes');
const taskRoutes = require('./routes/taskRoutes');
const app = express();
app.use(express.json());
app.use(cors());
mongoose.connect(process.env.MONGO_URI, {
```

```
    useUrlParser: true,  
    useUnifiedTopology: true  
  }).then(() => console.log("MongoDB Connected"))  
  .catch(err => console.log(err));  
App.use('/api/auth', authRoutes);  
App.use('/api/tasks', taskRoutes);  
Const PORT = process.env.PORT || 5000;  
App.listen(PORT, () => console.log(` Server running on port ${PORT} ` ));
```

1.2 models/User.js (User Model for Authentication)

```
const mongoose = require('mongoose');  
const UserSchema = new mongoose.Schema({  
  name: String,  
  email: { type: String, unique: true },  
  password: String  
});  
Module.exports = mongoose.model('User', UserSchema);
```

1.3 models/Task.js (Task Model for CRUD Operations)

```
const mongoose = require('mongoose');  
const TaskSchema = new mongoose.Schema({  
  title: String,  
  description: String,  
  category: String,  
  status: { type: String, enum: ['pending', 'completed'], default: 'pending' },  
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }  
});  
Module.exports = mongoose.model('Task', TaskSchema);
```

1.4 routes/authRoutes.js (Authentication Routes)

```

const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/User');
const router = express.Router();

router.post('/register', async (req, res) => {
  const { name, email, password } = req.body;
  const hashedPassword = await bcrypt.hash(password, 10);
  const newUser = new User({ name, email, password: hashedPassword });
  try {
    await newUser.save();
    res.status(201).json({ message: 'User registered successfully' });
  } catch (error) {
    Res.status(500).json({ message: 'Error registering user' });
  }
});

Router.post('/login', async (req, res) => {
  Const { email, password } = req.body;
  Const user = await User.findOne({ email });
  If (!user || !await bcrypt.compare(password, user.password)) {
    Return res.status(401).json({ message: 'Invalid credentials' });
  }
  Const token = jwt.sign({ userId: user._id }, process.env.JWT_SECRET, { expiresIn: '1h' });
  Res.json({ token, userId: user._id });
});

Module.exports = router;

```

1.5 routes/taskRoutes.js (Task CRUD Routes)

```

const express = require('express');
const Task = require('../models/Task');
const router = express.Router();

// Create a Task
Router.post('/', async (req, res) => {
  const { title, description, category, userId } = req.body;
  const task = new Task({ title, description, category, userId })
  try {
    await task.save();
    res.status(201).json(task);
  } catch (error) {
    res.status(500).json({ message: 'Error creating task' });
  }
});

// Get All Tasks
Router.get('/:userId', async (req, res) => {
  const tasks = await Task.find({ userId: req.params.userId });
  res.json(tasks);
});

// Update a Task
Router.put('/:id', async (req, res) => {
  const { title, description, category, status } = req.body;
  try {
    const task = await Task.findByIdAndUpdate(req.params.id, { title, description,
category, status }, { new: true });
    res.json(task);
  }

```

```

    } catch (error) {
      Res.status(500).json({ message: 'Error updating task' });
    }
  });
// Delete a Task
Router.delete('/:id', async (req, res) => {
  Try {
    Await Task.findByIdAndDelete(req.params.id);
    Res.json({ message: 'Task deleted' });
  } catch (error) {
    Res.status(500).json({ message: 'Error deleting task' });
  }
});
Module.exports = router;

```

2.Frontend (React.js)

- Install Dependencies

Npx create-react-app frontend

Cd frontend

Npm install axios react-router-dom bootstrap

Create Frontend Files

1.1 src/api.js (API Configuration)

```

import axios from 'axios';

const API_URL = 'http://localhost:5000/api';

export default axios.create({ baseURL: API_URL });

```

1.2 src/components/Auth.js (Login & Register)

```

import React, { useState } from 'react';
import axios from '../api';

export default function Auth({ setToken }) {

  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const login = async () => {

    const res = await axios.post('/auth/login', { email, password });
    localStorage.setItem('token', res.data.token);
    setToken(res.data.token);
  };

  Return (
    <div>
      <h2>Login</h2>
      <input type="email" onChange={(e) => setEmail(e.target.value)}
placeholder="Email" />
      <input type="password" onChange={(e) => setPassword(e.target.value)}
placeholder="Password" />
      <button onClick={login}>Login</button>
    </div>
  );
}

```

1.3 src/components/TaskManager.js (CRUD Task Management)

```

import React, { useEffect, useState } from 'react';
import axios from '../api'

export default function TaskManager() {

  const [tasks, setTasks] = useState([]);

```

```
const userId = localStorage.getItem('userId')

useEffect(() => {
  axios.get(`/tasks/${userId}`).then(res => setTasks(res.data));
}, []);

Return (
  <div>
    <h2>Task Manager</h2>
    {tasks.map(task => <div key={task._id}>{task.title}</div>)}
  </div>
);
}
```